

Tipi di Dati Composti

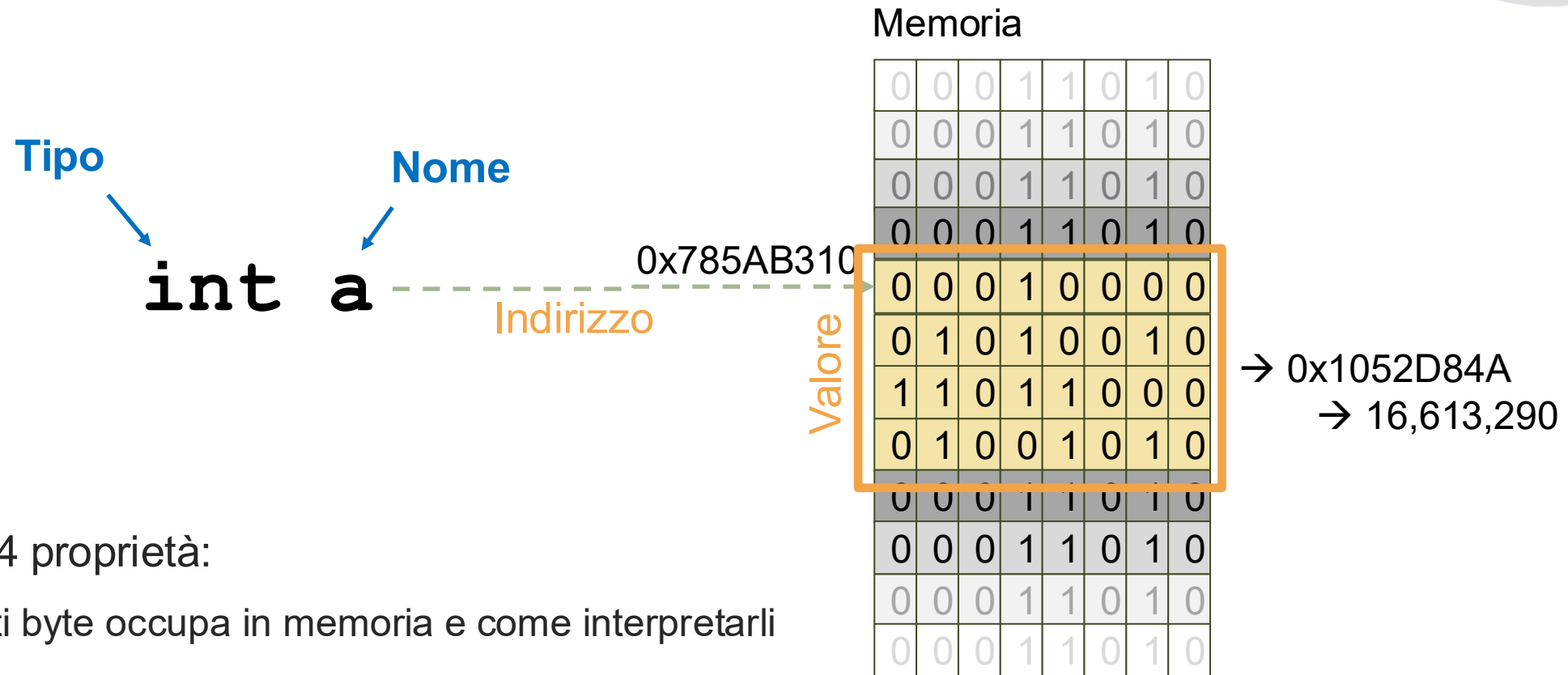
`struct, typedef, enum, union`

Questi lucidi sono basati sui lucidi di Programmazione 2 del Prof. Damiani e della Prof.ssa Baroglio



Variabili

Variabili in C



- Variabili in C hanno 4 proprietà:
 - **Tipo**: indica quanti byte occupa in memoria e come interpretarli
 - **Nome**: identifica la variabile per il programmatore
 - **Indirizzo**: dove si trova in memoria
 - **Valore**: il contenuto nell'area di memoria associato alla variabile

-
- Diagram illustrating the difference between L-value and R-value:
- L-VALUE (Left):** Points to the variable `X`. Below `X` is the text "Indirizzo di X" (Address of X).
 - R-VALUE (Right):** Points to the expression `X + 1`. Below `X` is the text "Valore di X" (Value of X).

Esempio I

```
int x;  
x = 0;  
x = x + 1;
```

- Eseguiamo queste tre istruzioni indicando cosa succede in memoria

Memoria Principale

0	0	0	1	1	0	1	0
0	0	0	1	1	0	1	0
0	0	0	1	1	0	1	0
0	0	0	1	1	0	1	0
0	0	0	1	0	0	0	0
0	1	0	1	0	0	1	0
1	1	0	1	1	0	0	0
0	1	0	0	1	0	1	0
0	0	0	1	1	0	1	0
0	0	0	1	1	0	1	0
0	0	0	1	1	0	1	0
0	0	0	1	1	0	1	0
0	0	0	1	1	0	1	0

Esempio II

```
int x;
```

```
x = 0;
```

```
x = x + 1;
```

- La dichiarazione alloca una memoria di 4 bytes (tipo `int`) in memoria principale e assegna a `x` l'indirizzo di partenza

Indirizzo:

0x785AB310 →

Memoria Principale

0	0	0	1	1	0	1	0
0	0	0	1	1	0	1	0
0	0	0	1	1	0	1	0
0	0	0	1	1	0	1	0
0	0	0	1	0	0	0	0
0	1	0	1	0	0	1	0
1	1	0	1	1	0	0	0
0	1	0	0	1	0	1	0
0	0	0	1	1	0	1	0
0	0	0	1	1	0	1	0
0	0	0	1	1	0	1	0
0	0	0	1	1	0	1	0

int → 4 bytes

Valore iniziale:

- bit casuali



Esempio III

```
int x;  
x = 0;  
x = x + 1;
```

- L'assegnamento cambia il valore contenuto nella porzione di memoria.
- L'indirizzo rimane invariato.

Indirizzo:

0x785AB310 →

Memoria Principale

0	0	0	1	1	0	1	0
0	0	0	1	1	0	1	0
0	0	0	1	1	0	1	0
0	0	0	1	1	0	1	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	1	1	0	1	0
0	0	0	1	1	0	1	0
0	0	0	1	1	0	1	0
0	0	0	1	1	0	1	0

Bit cambiati

Esempio IV

```
int x;  
x = 0;  
x = x + 1;
```

- La modifica si basa sul valore precedente.
- L'indirizzo è sempre invariato.

Indirizzo:

0x785AB310 →

Memoria Principale

0	0	0	1	1	0	1	0
0	0	0	1	1	0	1	0
0	0	0	1	1	0	1	0
0	0	0	1	1	0	1	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	1
0	0	0	1	1	0	1	0
0	0	0	1	1	0	1	0
0	0	0	1	1	0	1	0
0	0	0	1	1	0	1	0

Bit cambiati

Big vs Little Endian

```
int x;
x = 0;
x = x + 0x0A0B0C0D;
```

- Come viene salvato 0x0A0B0C0D in memoria?
 - Big Endian: dal byte più significativo (Motorola)
 - Little Endian: dal byte meno significativo (Intel)

Indirizzo Memoria Principale

0x10	0	0	0	1	1	0	1	0	
0x11	0	0	0	1	1	0	1	0	
0x12	0	0	0	1	1	0	1	0	
0x13	0	0	0	1	1	0	1	0	
0x14	0	0	0	0	1	0	1	0	0x0A
0x15	0	0	0	0	1	0	1	1	0x0B
0x16	0	0	0	0	1	1	0	0	0x0C
0x17	0	0	0	0	1	1	0	1	0x0D
0x18	0	0	0	1	1	0	1	0	
0x19	0	0	0	1	1	0	1	0	
0x1A	0	0	0	1	1	0	1	0	
0x1B	0	0	0	1	1	0	1	0	

Big Endian

Memoria Principale

0	0	0	1	1	0	1	0	
0	0	0	1	1	0	1	0	
0	0	0	1	1	0	1	0	
0	0	0	1	1	0	1	0	
0	0	0	0	1	1	0	1	0x0D
0	0	0	0	1	1	0	0	0x0C
0	0	0	0	1	0	1	1	0x0B
0	0	0	0	1	0	1	0	0x0A
0	0	0	1	1	0	1	0	
0	0	0	1	1	0	1	0	
0	0	0	1	1	0	1	0	
0	0	0	1	1	0	1	0	

Little Endian

Indirizzo $\leftrightarrow$ Valore: Operatori $\&$ e $*$

- $\& \text{ x}$: $\&$ indica l'indirizzo della variabile x
- $* \text{ ptr}$: $*$ deferenzia un'espressione, cioè permette di accedere al valore contenuto all'indirizzo calcolato da ptr
- Esempio particolare:
 - $* (\& \text{ x})$?
 - $*$ deferenzia l'indirizzo di $\& \text{ x}$, cioè il valore di x

struct

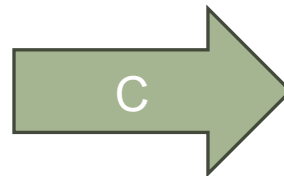
Strutture I

- **struct:**
 - in C definisce una collezione di dati correlati che possono essere disomogenei
 - Sono tipi di dati derivati; sono costruiti utilizzando tipi esistenti
 - I membri di una **struct** possono avere tipi distinti
 - Esempio:

⚠ Array: collezione di oggetti omogenei

LIBRO:

titolo: stringa
num. pagine: intero
autore: stringa
prezzo: numero con virgola



```
struct {  
    char titolo[MAXT];  
    int pagine;  
    char autore[MAXN];  
    double prezzo;  
};
```

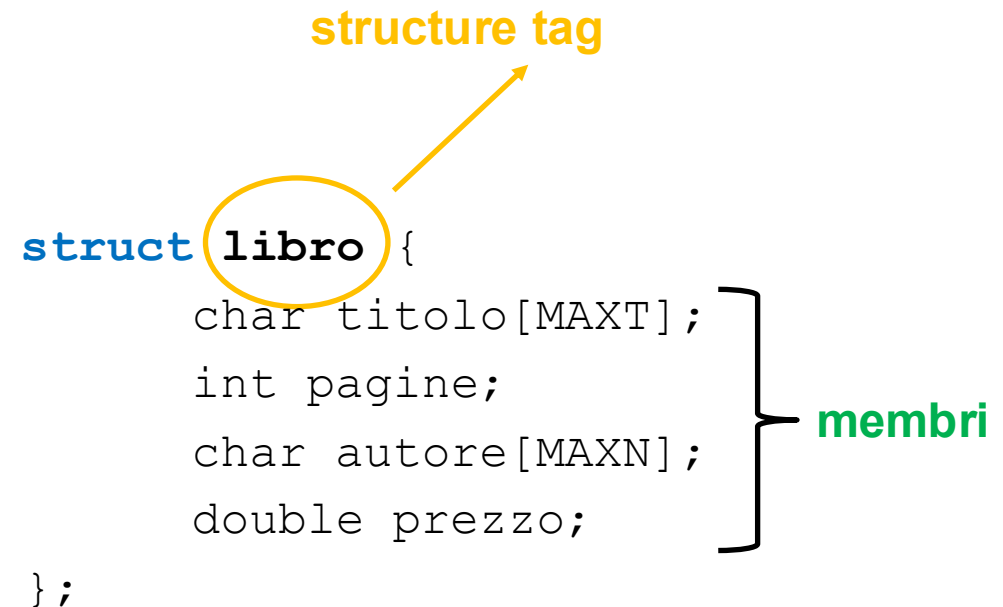
Strutture II

- La definizione di **struct**
 - Introduce un tipo
 - Non alloca memoria
- È costituita da:
 - keyword: **struct**
 - L'etichetta della struttura (**structure tag**)
 - Può essere facoltativa
 - Vari **membri** dichiarati come variabili tra le parentesi graffe

```
struct libro {  
    char titolo[MAXT];  
    int pagine;  
    char autore[MAXN];  
    double prezzo;  
};
```

structure tag

membri



Strutture III

- È possibile definire una `struct` e dichiarare delle variabili insieme

```
struct libro {  
    char titolo[MAXT];  
    int pagine;  
    char autore[MAXN];  
    double prezzo;  
}; /* definizione */  
  
struct libro L; /* dichiarazione */
```

structure tag non necessaria



```
struct {  
    char titolo[MAXT];  
    int pagine;  
    char autore[MAXN];  
    double prezzo;  
} L ; /* definizione e dichiarazione */
```



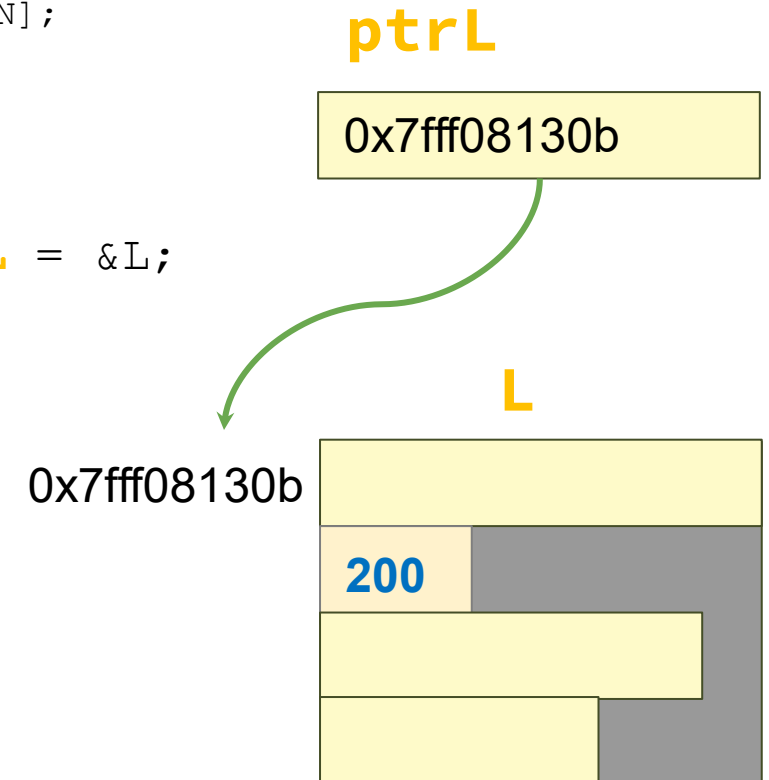
Accesso ai Campi: Operatori . e ->

- Operatore . membro della struct
accede ai membri
 - Esempio: `L.pagine = 200`
- Operatore -> puntatore alla struttura
accede ai membri tramite puntatore
 - Esempio: `ptrL->pagine`
 - Equivale a: `(*ptrL).pagine`
- Convenzione di stile
 - Niente spazi attorno agli operatori . e ->

```
struct libro {
    char titolo[MAXT];
    int pagine;
    char autore[MAXN];
    double prezzo;
};

struct libro L;
struct libro* ptrL = &L;

L.pagine = 200;
```

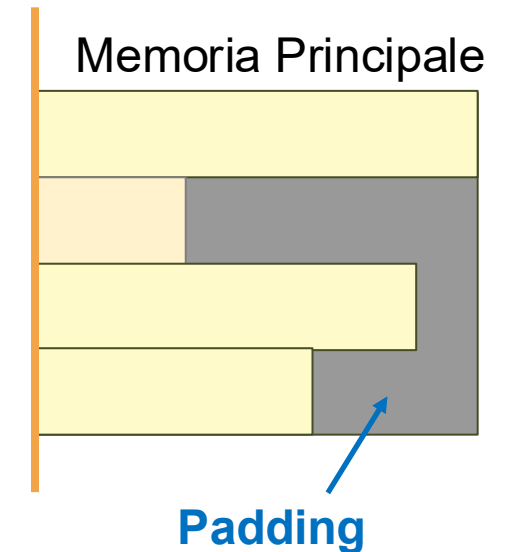
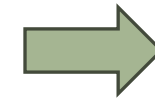


Strutture in Memoria: Padding

- Membri della `struct`:
 - Allocati in ordine di specifica
 - Hanno dimensioni diverse
 - Spesso allocati in modo da essere allocato sul confine della parola usata dall'architettura hardware
 - Spesso 4 byte (32 bit)
 - Migliora efficienza accessi in memoria*
- Conseguenza:
 - **Padding**: una `struct` può avere allocata un po' più memoria dello stretto necessario

```
struct libro {  
    char titolo[MAXT];  
    int pagine;  
    char autore[MAXN];  
    float prezzo;  
};
```

`struct libro L;`



Allineati
alla parola

Padding

*Piu dettagli ad Architettura

Padding: Conseguenze

```
struct prova {  
    char a;  
    char b;  
    int  c;  
};
```

```
struct prova P1, P2;
```

P1 = P2  // Ok

(P1 == P2)  // Non si può fare

- Assegnamento:
 - P1 = P2
 - Va bene perché copiamo tutta la memoria
- Confronto:
 - (P1 == P2)
 - Non si può fare.
 - Perché?
 - **Padding** contiene bit casuali

Strutture Autoreferenziali

```
struct employee {  
    char firstname[20];  
    char lastname[20];  
    struct employee manager;  
};
```



```
struct employee {  
    char firstname[20];  
    char lastname[20];  
    struct contact phone;  
};
```



```
struct employee {  
    char firstname[20];  
    char lastname[20];  
    struct employee* managerPtr;  
};
```



- Una `struct`
 - Non può contenere un membro del proprio tipo di `struct`
 - Perché?
 - Causerebbe una ricorsione infinita
 - Può contenere `struct` di altri tipi
 - Può invece contenere un puntatore a quel tipo di `struct`
 - **Strutture autoreferenziali**

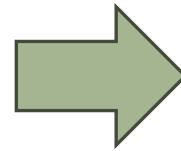
Un Esempio più Complesso I

- Possiamo modellare dati complessi
- Esempio:
 - Iniziamo definendo **dichiarazione nutrizionale** di un alimento

```
struct dich_nutriz {  
    int  energia;  
    float  grassi;  
    float  carboidrati;  
    float  fibre;  
    float  proteine;  
    float  sale;  
}
```

Un Esempio più Complesso II

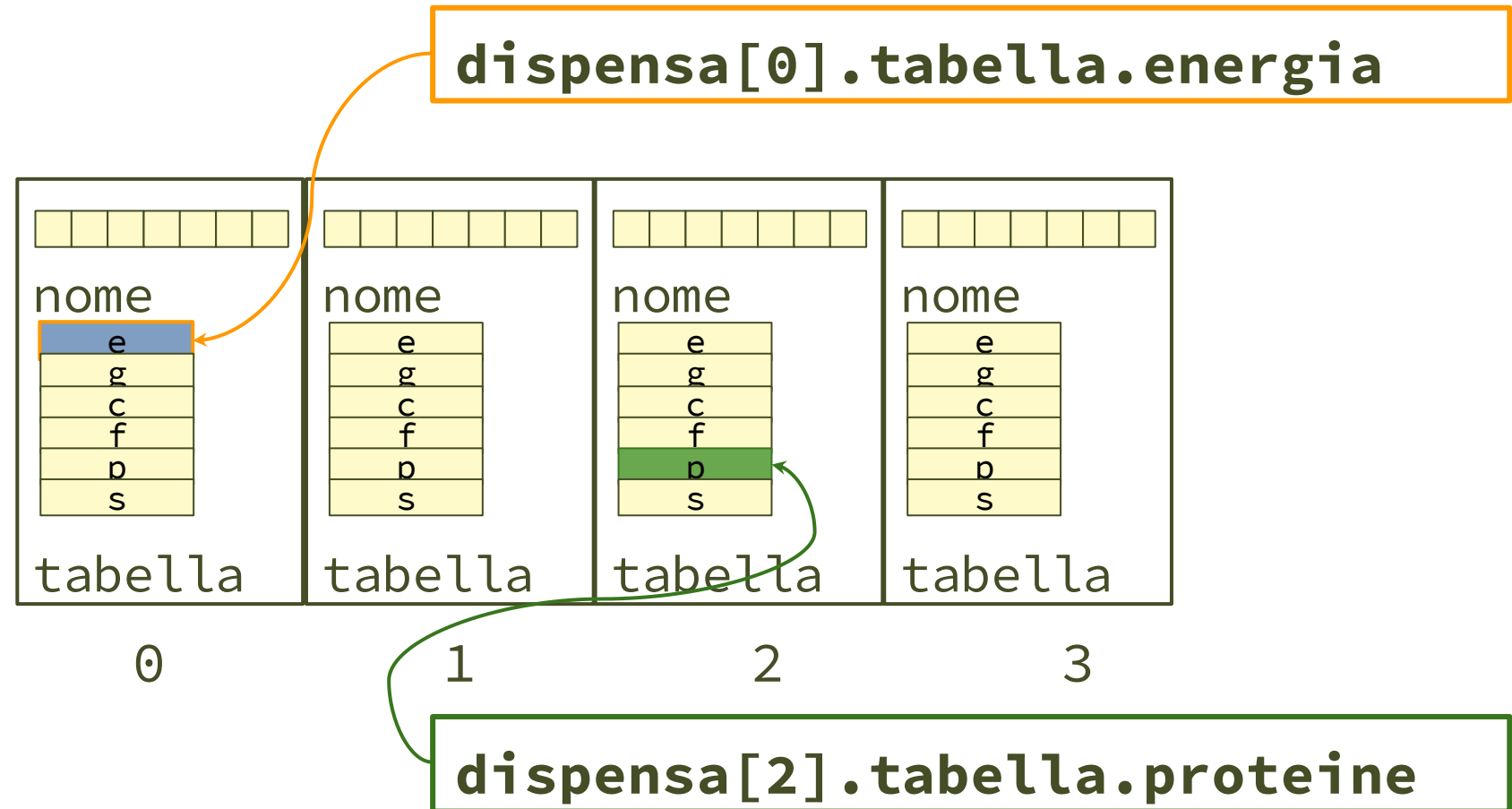
- Esempio:
 - In **dispensa** ci sono tanti cibo ognuno con la sua **dichiarazione nutrizionale**
- Posso modellarlo come:
 - **Array di struct** cibo, ognuna comprende
 - un nome
 - una dichiarazione nutrizionale
 - **struct annidata**



```
struct cibo {  
    char nome[MAX];  
    struct dich_nutriz tabella;  
} dispensa[MAXD];
```

Un Esempio più Complesso III

- Uso la conoscenza della struttura per accedere all'informazione e che mi serve



Strutture: Cheatsheet

```
libro L = {"Manuale C", 100, "C. Rossi", 9.90}; // inizializzazione
```

```
libro L = {}; // inizializza tutti i membri a 0 o (NULL per puntatori)
```

```
L = L2; // assegnazione di una struct a una struct
```

```
libro s[MAX]; // array di struct
```

```
L.pagine = 200; // accesso a un campo della struct
```

```
ss[2].autore // campo autore del III libro di ss
```

```
libro *s1; // puntatore a struct libro
```

```
s1->pagine = 200; // accesso a un campo dal puntatore a struct
```

```
libro *sd = (libro *) malloc(sizeof(libro)*N);
```

```
// allocazione dinamica di array di N libri
```

```
sd[2].autore // campo autore del III libro di sd
```



```
if (L1 == L2) // NON SI PUÒ FARE
```

Utilizzo con le Funzioni I

- Le variabili `struct` sono passate per valore
 - Alla stessa maniera dei tipi primitivi, e.g., `int`, `float`, ...
- I membri della `struct` vengono passati come variabili dello stesso tipo
 - Pertanto la funzione chiamata non modifica la struct nel chiamante
 - Per modificare la `struct` bisogna passarla per riferimento (`&`)
- Passare le strutture per riferimento è più efficiente
 - Non occorre la copiatura dell'intera struttura

Utilizzo con le Funzioni II

- **Esempio:**

```
void stampalibro(struct libro L){  
  
    // richiede la copia di sizeof(struct libro) bytes!  
  
}  
  
void stampalibro(struct libro* Lptr){  
  
    // molto più efficiente  
  
}
```

```
struct libro {  
    char titolo[MAXT];  
    int pagine;  
    char autore[MAXN];  
    float prezzo;  
};
```


Utilizzo con le Funzioni: Esercizio

- Provate a compilare ed eseguire:
 - struct.c
- In locale o online:
 - <https://wandbox.org/>

```
#include <stdio.h>
#include <stdlib.h>
#define N 3

/* definiamo due strutture che contengono un array */
struct sa {
    int a[N]; /* array statico */
};
struct sp {
    int * p; /* puntatore (array dinamico) */
};

/* ... e alcune funzioni di test */
void test0(struct sa str) {
    str.a[0] = 7;
}

void test1(int *arr) {
    arr[1] = 8;
}

void test2(int arr[3]) {
    arr[2] = 9;
}

void test3(struct sp str) {
    str.p[0] = 11;
}

int main(void)
{
    struct sa myst1;
    struct sp myst2;
    /* allochiamo la memoria dinamica */
    myst2.p = malloc(sizeof(int) * N);

    /* Inizializziamo tutti gli elementi di entrambi gli array a -1 */
    for (int i = 0; i < N; i++) {
        myst1.a[i] = myst2.p[i] = -1;
    }

    test0(myst1);
    printf("Hello0 : %d\n", myst1.a[0]); // prints -1

    test1(myst1.a);
    printf("Hello1 : %d\n", myst1.a[1]); // prints 8

    test2(myst1.a);
    printf("Hello2 : %d\n", myst1.a[2]); // prints 9

    test3(myst2);
    printf("Hello3 : %d\n", myst2.p[0]); // prints 11
}
```

typedef

typedef

- **typedef** permette di creare sinonimi (o alias) per tipi precedentemente definiti
 - Non crea un nuovo tipo: crea un nome alternativo per un tipo esistente
- Stile:
 - Per convenzione si scrive in Maiuscolo la prima lettera dei nomi degli alias

- Esempio uso:

```
typedef char* Stringa;
```

Una volta definita, si può:

```
Stringa s;  
Stringa doc[MAXLINES];  
int strcmp(Stringa, Stringa);
```



typedef

- Spesso usato per accorciare i nomi delle `struct`
 - Esempio:

```
typedef struct {  
    char titolo[MAXT];  
    int pagine;  
    char autore[MAXN];  
    float prezzo;  
} Libro;           // nuovo tipo “Libro”  
  
Libro L1, L2;      // variabili di tipo “Libro”
```

⚠ La structure tag
non è più necessaria



enum

enum

- **enum** definisce un insieme di valori interi costanti convenientemente rinominati con etichette (univoche)
 - Sono un mezzo efficiente per associare valori interi a nomi
 - Preferibile alle `#define` (compilatore controlla se valore fa parte del gruppo)
 - Se non diversamente specificato i valori iniziano da 0
 - Più etichette possono avere lo stesso valore costante
- Esempio:

```
enum months { JAN = 1, FEB, MAR, APR, MAY, JUN, JUL, AUG, SEP, OCT, NOV, DEC }; /* JAN è 1, FEB è 2, e così via */
```
- Convenzione di stile:
 - Le etichette si scrivono tutto in MAIUSCOLO



enum e typedef

- **typedef** e **enum** possono essere usati insieme

- Esempio:

```
typedef enum {OOM = -1, ALREADYPRESENT, INSERTED} Outcome;
```

- Outcome ora è un intero che può valere -1, 0, 1:

```
Outcome function() {  
    ...  
    return OOM;  
}
```



union

Unioni: cosa sono?

- Unioni:
 - Variabili **simili alle strutture**, ma che in un dato momento **contengono uno solo** dei membri specificati entro le parentesi graffe { }
- Sintassi **identica** a quella per le **struct**
 - Si sostituisce la keyword **struct** con la keyword **union**

```
union myunion {  
  
    char   cval;  
    int    ival;  
    float  fval;  
} u;
```

⚠ La variabile u può contenere:

- un CHAR di nome cval
- OPPURE**
- un INTERO di nome ival
- OPPURE**
- un FLOAT di nome fval

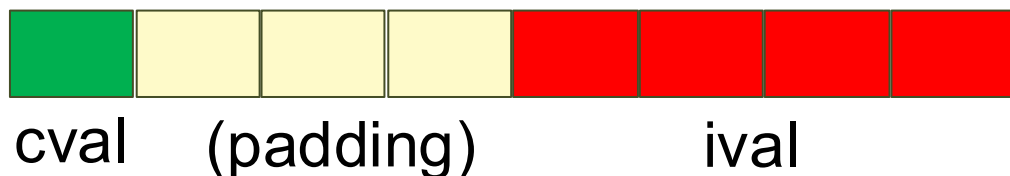


Unioni: Cosa sono?

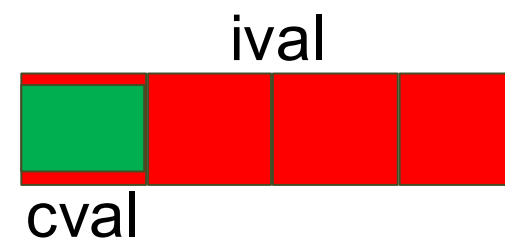
- Attenzione alla differenza tra **strutture** e **unioni**

```
struct mystruct {  
    char cval;  
    int ival;  
} s;
```

- In memoria:



```
union myunion {  
    char cval;  
    int ival;  
} u;
```



Unioni: Sintassi

- Stessa sintassi e operazioni delle strutture.

```
union myunion {  
    char cval;  
    int ival;  
    float fval;  
} u, *ptru;  
  
u.cval = 'h';  
  
u.ival = 0xDE4D7E91;    /* sostituisce cval! */  
  
ptru = &u;  
  
ptru->fval = 3.14;      /* sostituisce ival! */
```



Unioni e Strutture Annidate I

- Una **struct** può essere un membro di **union**

```
union tag1 {  
    struct tag2 {  
        int ival;  
        float fval;  
    } svar;  
}
```

```
    double d;  
} uvar;
```

```
/* accesso al membro interno della struttura */  
uvar.svar.i = 1;
```

Unioni e Strutture Annidate II

- Similmente, una **union** può apparire in una **struct**

```
struct {  
    int flags;  
    char *name;  
    int utype;  
  
    union {  
        int ival;  
        float fval;  
        char *sval;  
    } u;  
  
} symtab[NSYM];
```



Unioni: Cosa Contengono in un Dato Istante?

- È responsabilità del programmatore tenere traccia di quale sia il tipo attualmente registrato in una **union**
- Approccio comune: **tagged union** o labelled union
 - Uso di una **variabile di supporto** per memorizzare il tipo di memorizzato nell'unione
 - Ad esempio: un piccolo intero con valori definiti tramite enumerazione

```
enum Union_Tag {IS_INT, IS_CHAR};
```

```
struct TaggedUnion {  
    enum Union_Tag tag;  
    union {  
        int    i;  
        char   c;  
    } data;  
};
```

← Variabile di supporto

Unioni: In Breve



- Una **union** è in pratica una struttura
 - Nella quale tutti i membri hanno spiazzamento nullo rispetto alla base
 - La struttura è sufficientemente grande da contenere il membro più ampio, garantendo il corretto allineamento in memoria per tutti i tipi presenti nella union

Unioni: Esercizio



- Dato la tagged **union** definita qui accanto, scrivete:
 1. Una funzione che stampi a video il contenuto nella **union**
 - Usate il passaggio per riferimento
 2. Un main che testi la funzione
 - Sia con un `int` che con un `char`
- Cosa succede se si prova a estrarre da una **union** un tipo sbagliato?
- Scheletro esercizio su Moodle:
 - `union_es.c`

```
enum Union_Tag {IS_INT, IS_CHAR};

typedef struct {
    enum Union_Tag tag;
    union {
        int    i;
        char   c;
    } data;
} TaggedUnion;
```